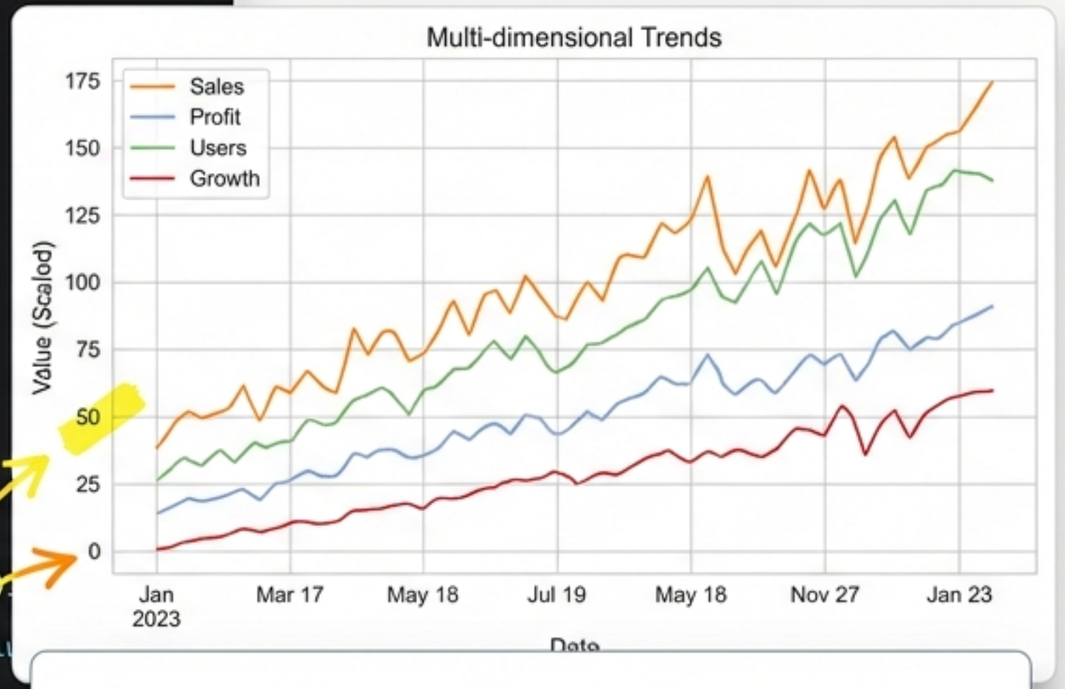
```python
# Import libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Set plot style
sns.set_theme(style="whitegrid")

# Generate sample data
def generate_data(n_points=100):
    # Simulate time-series data
    time = pd.date_range('2023-01-01', periods=n_points, freq='D')
    data = pd.DataFrame({
        'Date': time,
        'Sales': [50 + i*0.5 + (i%10)*2 for i in range(n_points)],
        'Profit': [20 + i*0.2 + (i%12)*1.5 for i in range(n_points)],
        'Users': [100 + i*1.1 + (i%15)*3 for i in range(n_points)],
        'Growth': [0.1 + i*0.005 + (i%20)*0.01 for i in range(n_points)]
    })
    return data.melt(id_vars='Date', var_name='Metric', value_name='Value')

df = generate_data()

# Create multi-line plot
plt.figure(figsize=(10, 6))
sns.lineplot(data=df, x='Date', y='Value', hue='Metric', palette='deep')

# Add titles and labels
plt.title('Multi-dimensional Trends')
plt.xlabel('Date')
plt.ylabel('Value (Scaled)')
plt.show()
```



Mastering Seaborn lineplots from basic trends to multi-dimensional grids

A progressive Code-to-Canvas visual walkthrough.

Preparing the environment and ingesting the raw dataset

```
# import drive
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
from google.colab import drive

drive.mount('/content/drive')
# Output: Mounted at /content/drive

student = pd.read_csv('/content/drive/MyDrive/Dataset/
sns_data.csv')
student
```

Library Toolkit

pandas / numpy
Core data manipulation engines.

matplotlib / seaborn
The primary visualization layers.

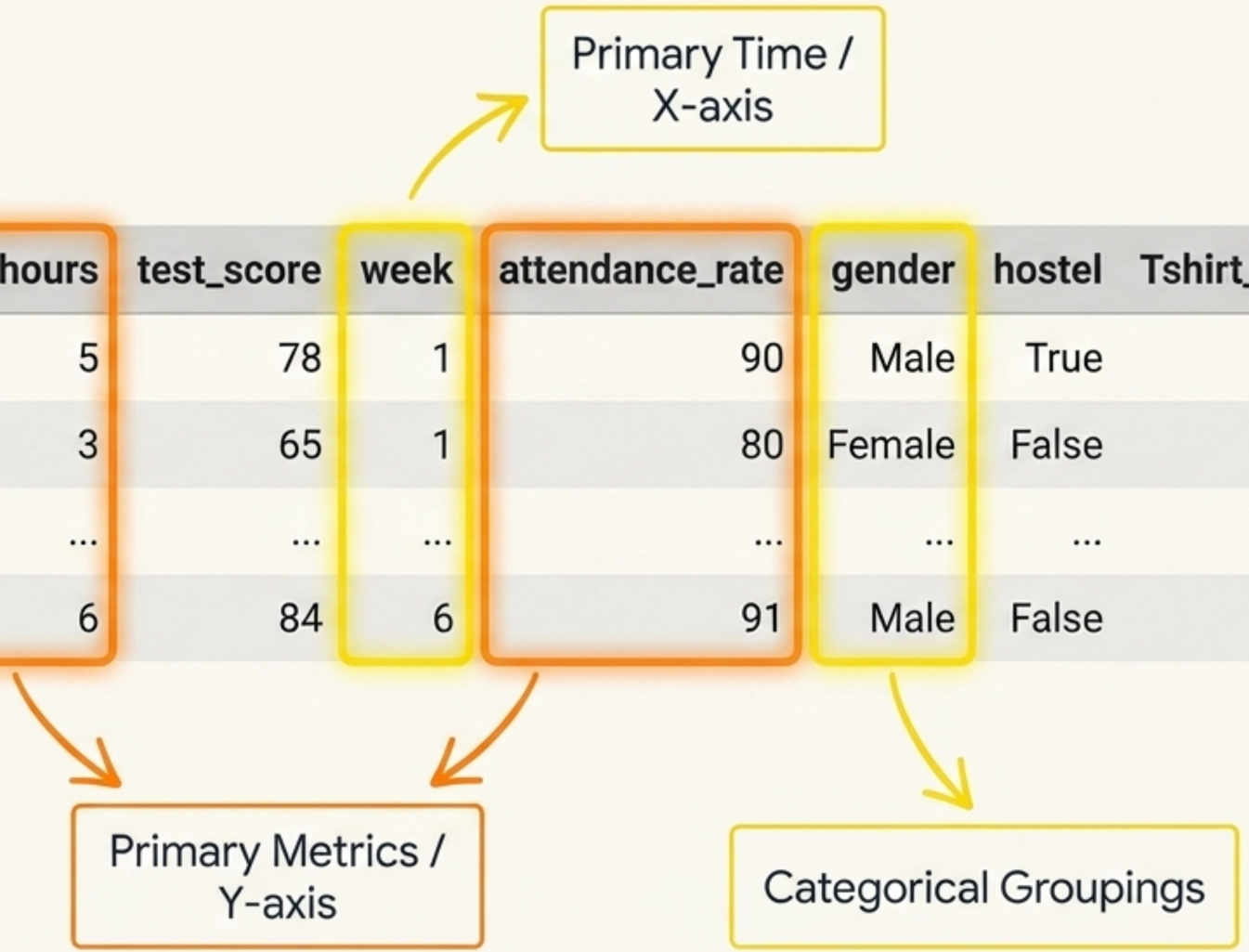
plotly.express
Alternative interactive plotting library.

google.colab
Environment utility required to mount the drive and access the raw sns_data.csv dataset.

Inspecting the structural anatomy of the student dataset

	student_id	class_level	subject	study_hours	test_score	week	attendance_rate	gender	hostel	Tshirt_size
1	0	Freshman	Math	5	78	1	90	Male	True	30
2	1	Freshman	Science	3	65	1	80	Female	False	20
3	...	...	...	...	...	...	...	...	...	...
4	94	Senior	Math	6	84	6	91	Male	False	30

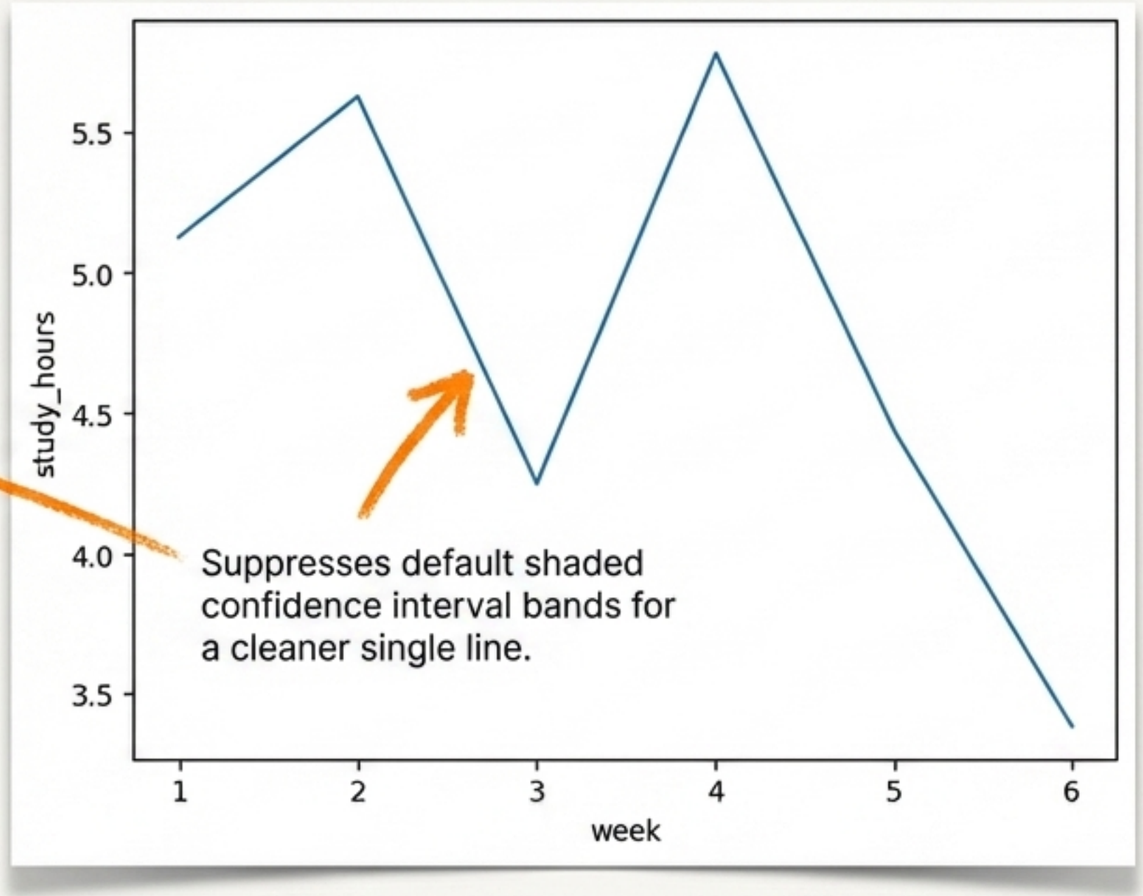
95 rows × 10 columns



Establishing a baseline single-trend lineplot

```
# axes level plot
sns.lineplot(data=student, x='week',
             y='study_hours',
             errorbar=None)
```

data=student: Ingests the pandas DataFrame.
x='week': Maps horizontal axis to time.
y='study_hours': Maps vertical axis to the target metric.

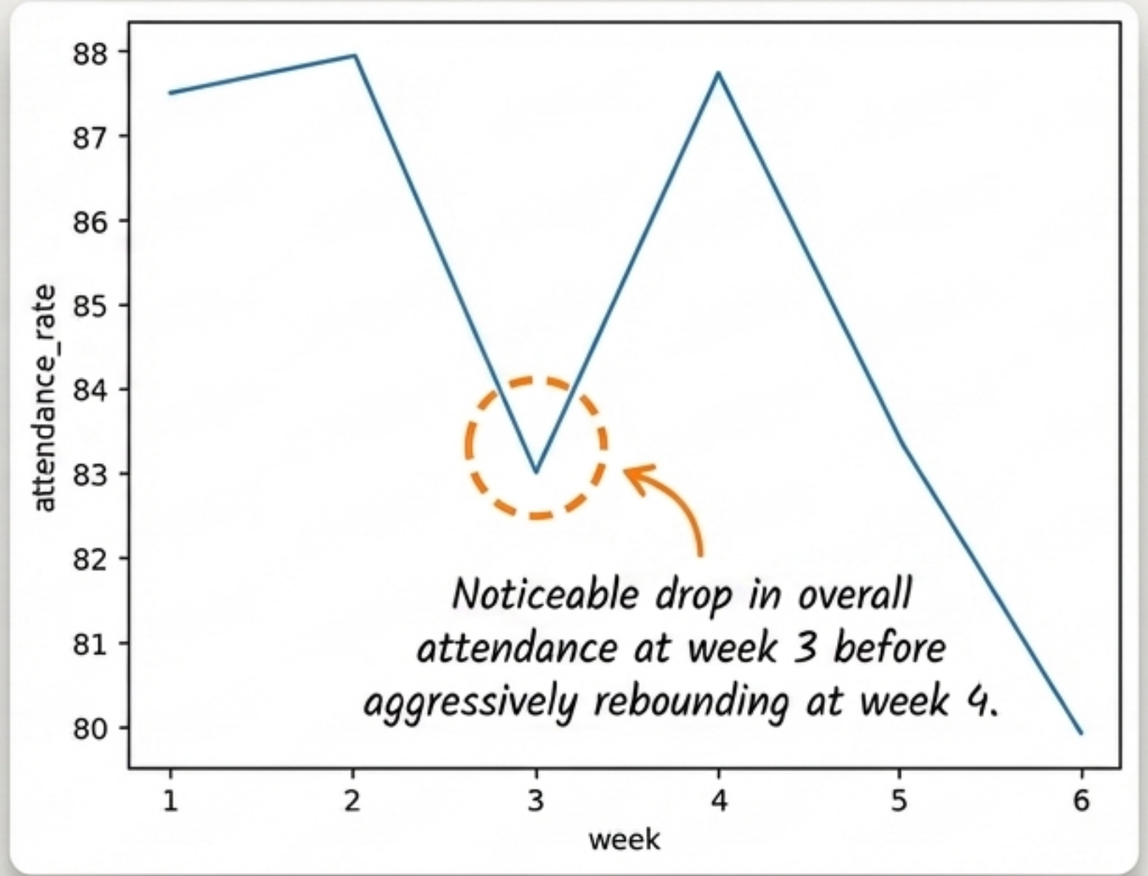


Output: <Axes: xlabel='week', ylabel='study_hours'>

Swapping the y-axis metric to reveal different trends

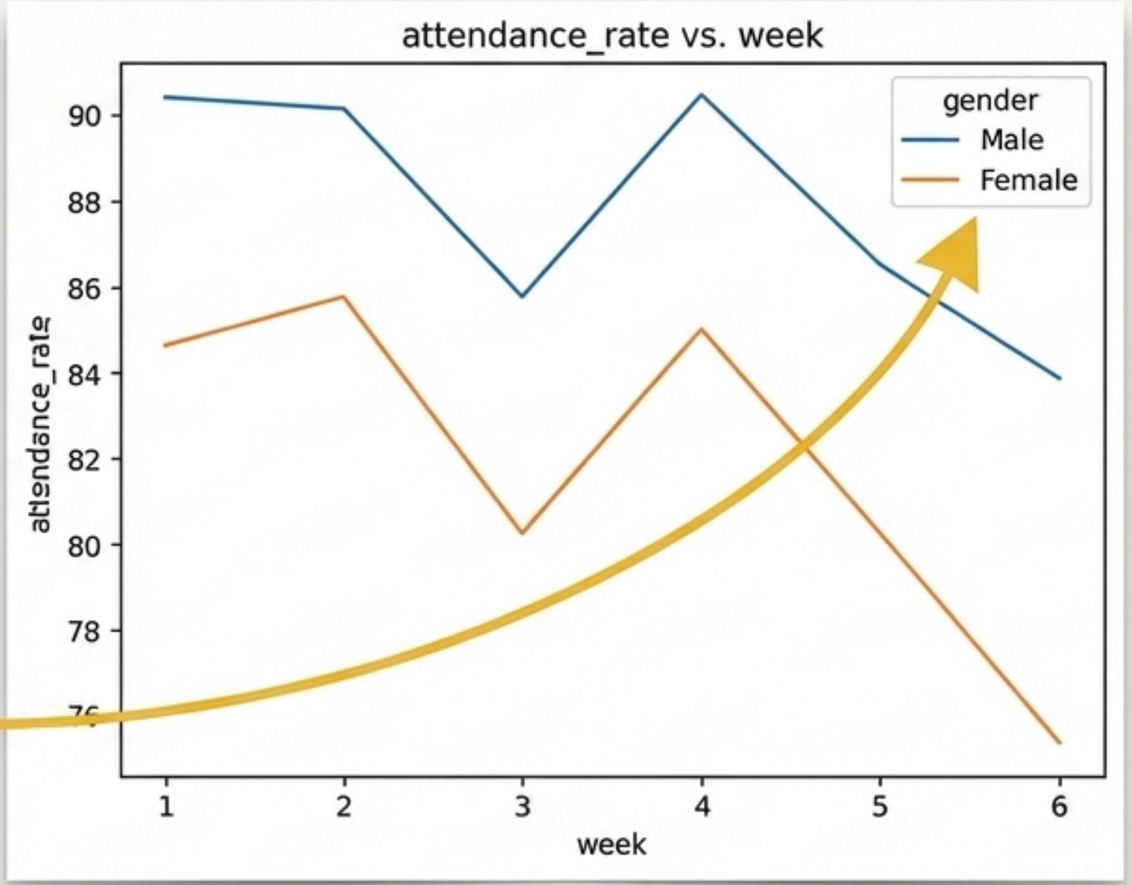
```
sns.lineplot(data=student,
    x='week',
    y='attendance_rate',
    errorbar=None)
```

Observation: By swapping the y argument from study_hours to attendance_rate, the underlying syntax remains identical but reveals a completely different dataset behavior.



Slicing data into multiple categories using the hue parameter

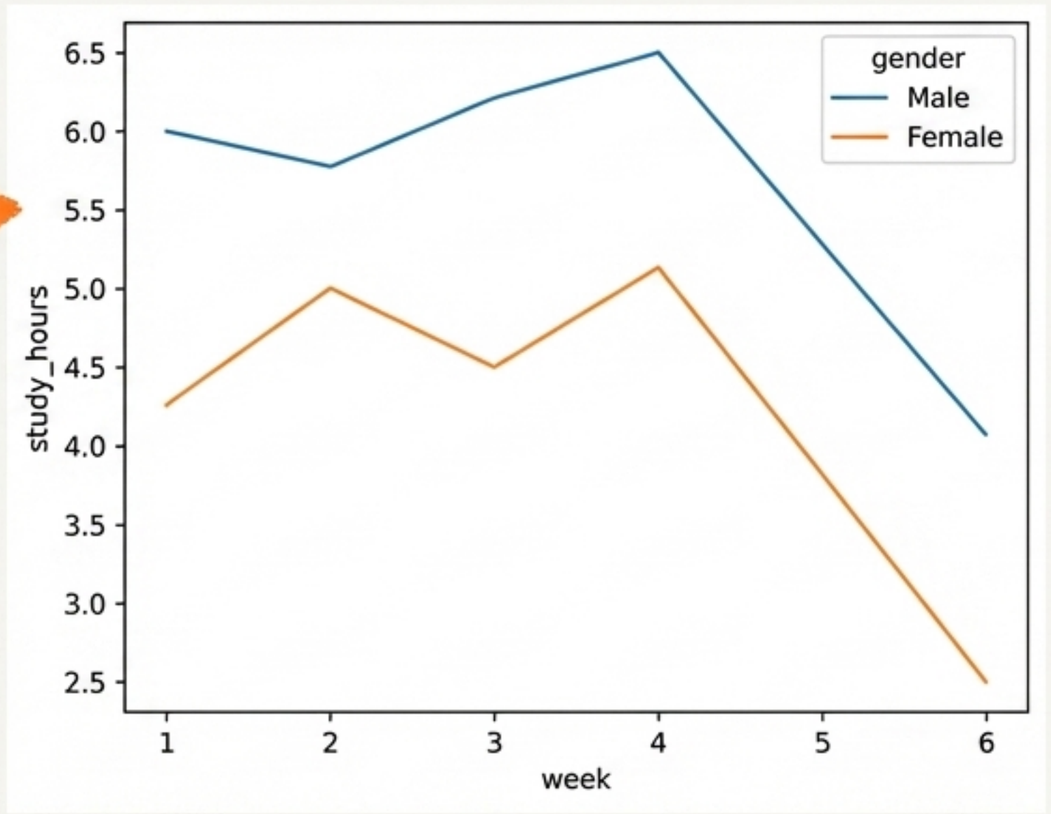
```
# we can use for see this details for sepecific attributes
sns.lineplot(data=student, x="week", y="attendance_rate",
             errorbar=None, hue="gender")
plt.show()



# we can use for see this details for sepecific attributes
sns.lineplot(data=student, x="week", y="attendance_rate",
             errorbar=None, hue="gender")
plt.show()
```



Male attendance consistently tracks roughly 5-6 points higher than Female attendance across all 6 weeks.

Applying consistent hue logic across different metrics

```
# we can use for see this details for sepecific attributes
sns.lineplot(data=student, x="week", y="study_hours",
             errorbar=None, hue="gender")
```




```
Output: <Axes: xlabel='week', ylabel='study_hours'>
```

Consistency: The syntax remains predictable. We apply the exact same grouping logic to observe how male and female students differ in their study hours instead of attendance.

Understanding the difference between Axes-level and Figure-level objects

Axes-Level Functions

`sns.lineplot`, `sns.scatterplot`

- **Mechanics:** Draws onto a single, specific `matplotlib.pyplot.Axes`.
- **Limitation:** Cannot easily generate multi-panel grid layouts organically.
- **Console Return:** Returns an `Axes` object (e.g., `<Axes: xlabel... >`).

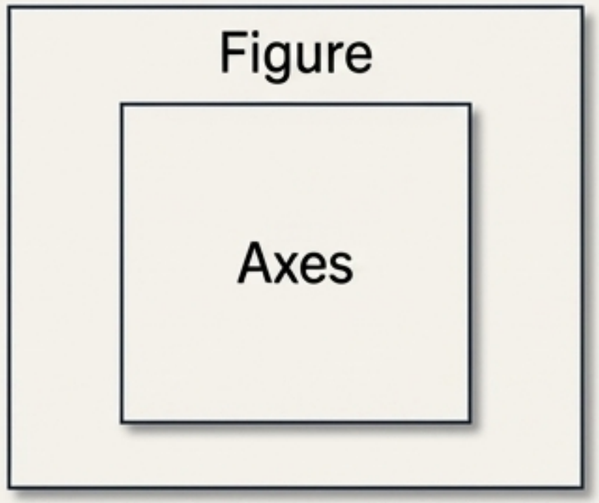
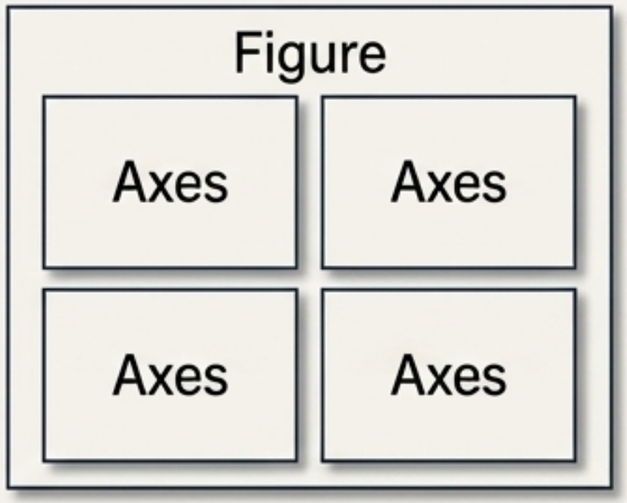


Figure-Level Functions

`sns.relplot`, `sns.catplot`

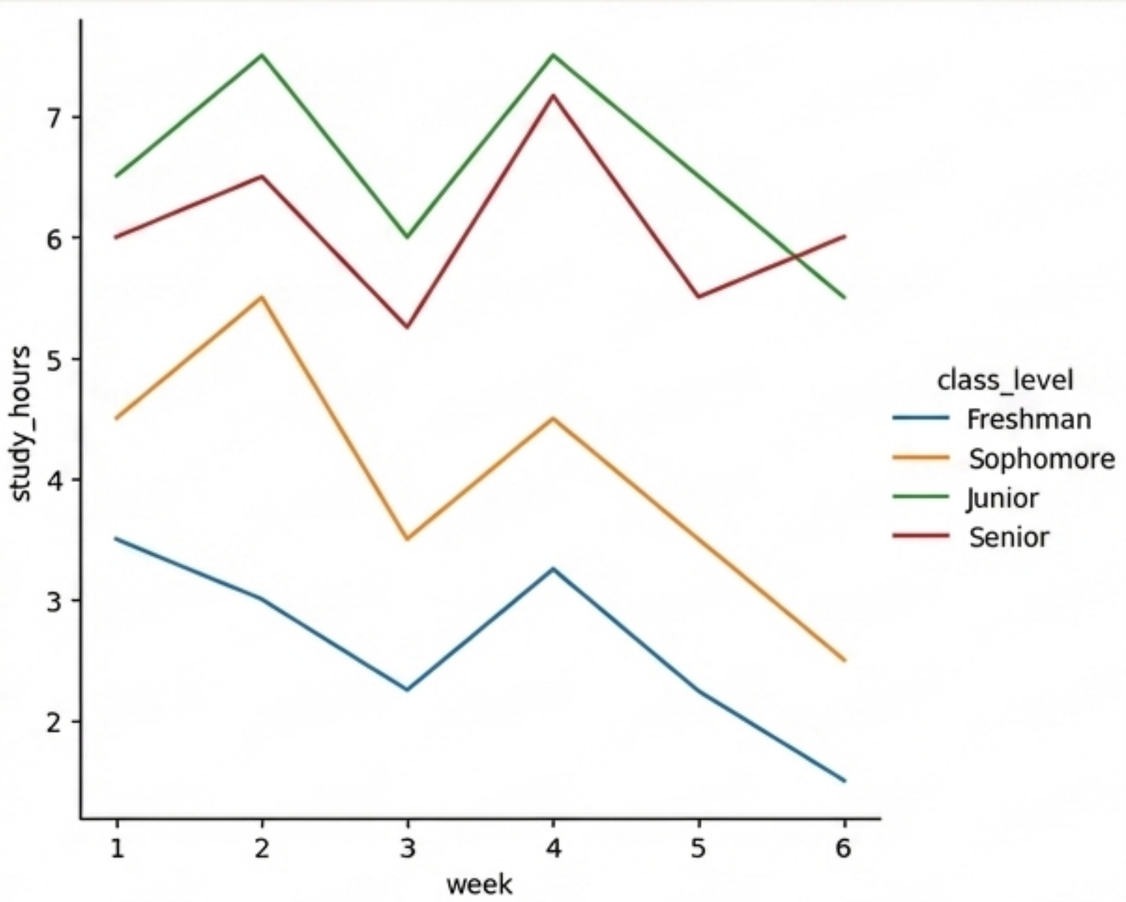
- **Mechanics:** Initializes a global `matplotlib.figure.Figure` capable of hosting multiple separate Axes simultaneously.
- **Advantage:** Designed specifically to create multi-plot grids via faceting parameters.
- **Console Return:** Returns a `FacetGrid` object (e.g., `<seaborn.axisgrid.FacetGrid....>`).



Upgrading to Figure-level plotting with relplot

```
# figure level: Here in figure has multiple axes level
data representation
sns.relplot(data=student, x="week", y="study_hours",
            hue="class_level", kind='line', errorbar=None)
```

Output: <seaborn.axisgrid.FacetGrid at 0x79f44891a630>

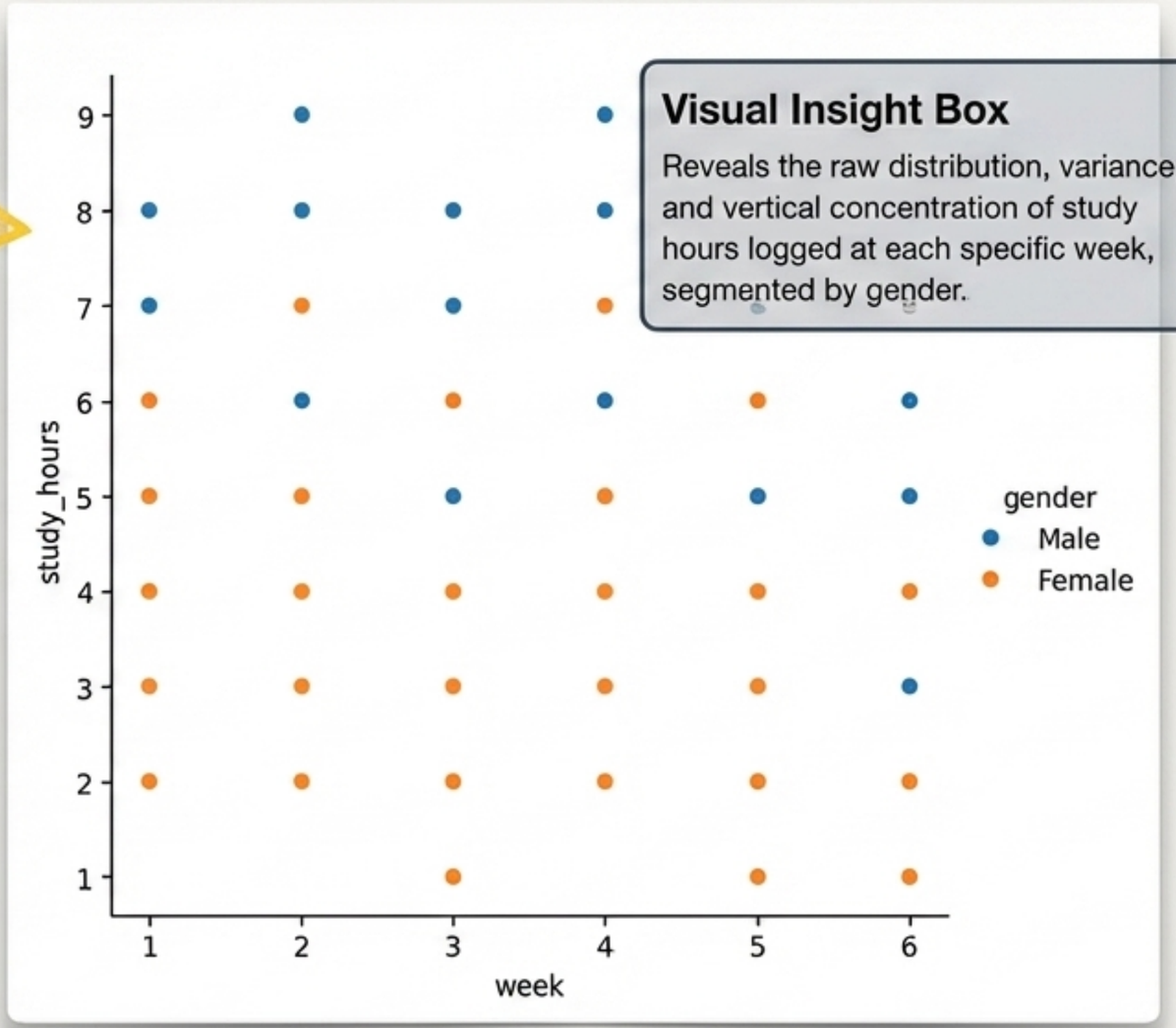


What Changed:

1. Function upgraded from `lineplot` to `relplot` (Relational Plot).
2. Added `kind='line'` to explicitly specify the visual representation type.
3. hue re-mapped to `class_level`, generating four distinct overlapping trend lines (Freshman, Sophomore, Junior, Senior).

Swapping visual geometry by altering the `kind` parameter

```
sns.relplot(data=student, x="week", y="study_hours",
            hue="gender", kind='scatter')
```



Visual Insight Box

Reveals the raw distribution, variance, and vertical concentration of study hours logged at each specific week, segmented by gender.

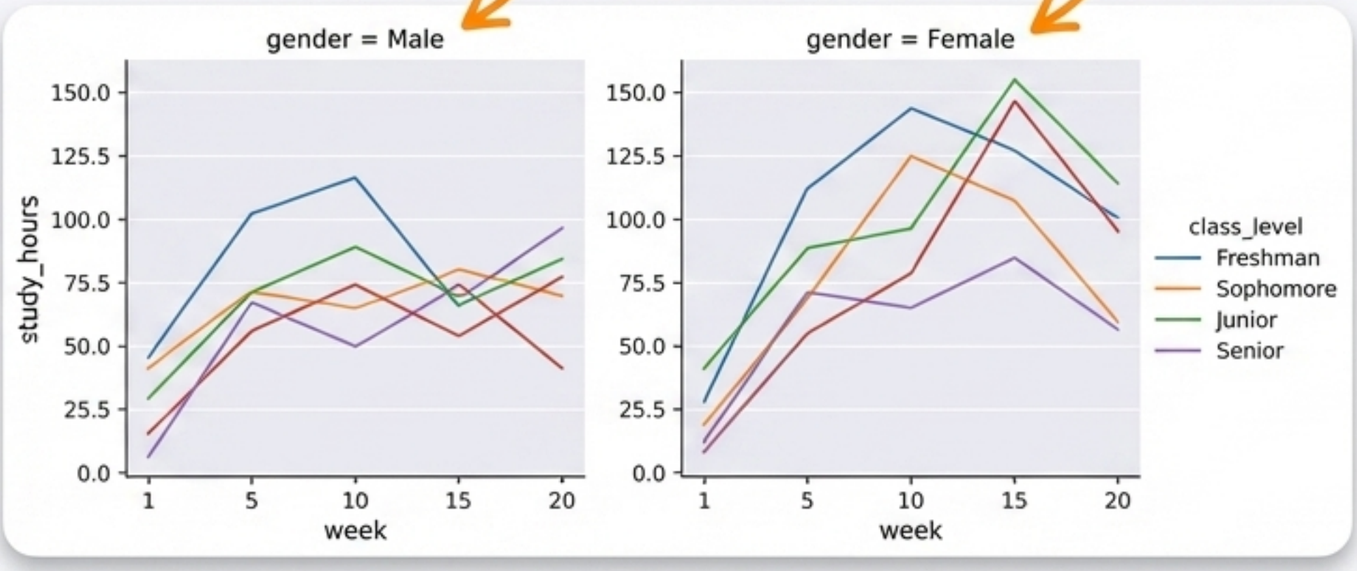
Syntax Note

Changing `kind='line'` to `kind='scatter'` completely transforms the underlying `FacetGrid` rendering. Notice that `errorbar=None` is dropped, as error bands do not apply to scatterpoint geometries.

Generating multi-plot comparative grids with a single parameter

```
sns.relplot(data=student, x="week", y="study_hours", errorbar=None, hue="class_level",
            kind='line', col='gender')
```

This single parameter instructs Seaborn to generate a separate, isolated subplot (column) for every unique value in the 'gender' category, instantly creating a side-by-side comparative dashboard.




```
<seaborn.axisgrid.FacetGrid at 0x79f44c8ebc20>
```

A quick-reference guide to core Seaborn lineplot parameters

Foundational Parameters

`data` : Specifies the ingest pandas DataFrame.

`x, y` : Maps target dataframe columns to the horizontal and vertical spatial axes.

Slicing & Styling Overrides

`hue` : Groups data by a specific category, rendering separate color-coded lines and a legend.

`errorbar=None` : Overrides library defaults to remove shaded confidence interval bands.

Figure-Level Control (relplot)

`kind` : Dictates the geometric rendering type ('line' or 'scatter').

`col` : Splits the primary dataset into entirely separate, side-by-side subplot columns based on categorical values.